

Guia de Pruebas con Jest

MyMDental - E-commerce Dental

Pruebas automatizadas con Jest + React Testing Library

Paleta de colores del proyecto:



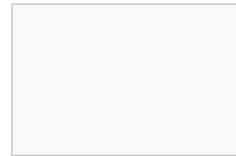
Cian (Acento)



Rosa (Detalles)



Oscuro (Texto)



Claro (Fondo)

Repositorio: <https://github.com/Duocantonio/mymdental>

Fecha de elaboracion: Mayo 2026

Version: 1.0

Indice

1. Introduccion
2. Requisitos del Sistema
3. Instalacion del Entorno de Pruebas
4. Estructura del Proyecto y Configuracion
5. Conceptos Fundamentales de Jest
6. Definicion de Funciones y APIs
7. Ejemplos de Pruebas (de Basico a Avanzado)
8. URLs de Prueba y Repositorio
9. Criterios de Aceptacion
10. Conclusion

1. Introduccion

Esta guia tiene como objetivo proporcionar un manual completo para la implementacion de pruebas automatizadas en el proyecto MyMDental, un e-commerce dental desarrollado con React + Vite. A lo largo de este documento, se explicara desde la instalacion del entorno de pruebas con Jest y React Testing Library, hasta la creacion de casos de prueba complejos que cubren interacciones de usuario, renderizado de componentes y flujos de navegacion.

Jest es un marco de pruebas (testing framework) desarrollado por Meta (Facebook) que se integra de manera nativa con proyectos React. Proporciona un entorno completo que incluye un corredor de pruebas (test runner), un sistema de aserciones (assertions), simulaciones (mocks) y herramientas de cobertura de codigo (code coverage).

Complementariamente, React Testing Library (RTL) proporciona utilidades para renderizar componentes React en un entorno simulado y realizar interacciones sobre ellos, fomentando buenas practicas como probar el comportamiento desde la perspectiva del usuario en lugar de probar detalles de implementacion interna.

IMPORTANTE: Todas las pruebas descritas en esta guia se ejecutan exclusivamente en el FRONTEND. No se requiere un servidor backend activo para las pruebas unitarias y de componentes, ya que se simulan las llamadas API mediante mocks.

2. Requisitos del Sistema

Sistema Operativo

El entorno de pruebas es multiplataforma y puede ejecutarse en los siguientes sistemas operativos:

- Windows 10 / 11 (64 bits)
- macOS 12+ (Monterey o superior)
- GNU/Linux (Ubuntu 20.04+, Fedora 38+, Debian 11+)

Software Requerido

Componente	Version Recomendada
Node.js	18.x LTS o superior
npm	9.x o superior
Git	2.30+
Navegador	Chrome/Edge/Firefox (ultima version)
Sistema	x64 (amd64)

Lenguaje y Tecnologias

Lenguaje principal: JavaScript (ES2022+). Los archivos de prueba utilizan JSX (JavaScript XML) para renderizar componentes React. El proyecto no utiliza TypeScript, por lo que las pruebas se escriben en JavaScript plano con JSX.

- Lenguaje: JavaScript (ECMAScript 2022+)
- Framework de pruebas: Jest 29+
- Testing library: @testing-library/react 14+
- Simulacion de navegador: jest-environment-jsdom
- Pruebas de eventos: @testing-library/user-event 14+

3. Instalacion del Entorno de Pruebas

Paso 1: Verificar Node.js y npm

Antes de comenzar, asegurese de tener Node.js instalado:

```
node --version
npm --version
```

Paso 2: Instalar dependencias del proyecto

Navegue hasta la raiz del proyecto e instale las dependencias existentes:

```
cd mymdental
npm install
```

Paso 3: Instalar Jest y React Testing Library

Ejecute el siguiente comando para instalar todas las dependencias de pruebas:

```
npm install --save-dev jest @testing-library/react @testing-library/jest-dom \
  @testing-library/user-event jest-environment-jsdom babel-jest \
  @babel/preset-env @babel/preset-react identity-obj-proxy
```

Paso 4: Configurar Babel

Cree el archivo babel.config.cjs en la raiz del proyecto:

```
module.exports = {
  presets: [
    ["@babel/preset-env", { targets: { node: "current" } }],
    ["@babel/preset-react", { runtime: "automatic" }],
  ],
};
```

Paso 5: Configurar Jest

Agregue la siguiente configuracion a su package.json:

```
"scripts": {
  "test": "jest",
  "test:watch": "jest --watch",
  "test:coverage": "jest --coverage"
},
"jest": {
  "testEnvironment": "jsdom",
  "moduleNameMapper": {
    "\\.(css|less|scss)$": "identity-obj-proxy",
    "\\.(jpg|jpeg|png|gif|svg)$": "<rootDir>/src/__mocks__/fileMock.js"
  },
  "setupFilesAfterEnv": ["<rootDir>/src/setupTests.js"],
  "transform": {
    "^.+\\.jsx?$": "babel-jest"
  }
}
```

Paso 6: Crear archivos de configuracion adicionales

Cree src/setupTests.js:

```
import "@testing-library/jest-dom";  
// Extiende expect con matchers personalizados  
// toBeInTheDocument, toHaveClass, toBeVisible, etc.
```

Cree src/__mocks__/fileMock.js:

```
module.exports = "test-file-stub";
```

Paso 7: Verificar la instalacion

Ejecute el comando para confirmar que todo funciona:

```
npx jest --version
```

Deberia ver la version de Jest (ej: 29.7.0).

4. Estructura del Proyecto y Configuracion

Las pruebas se organizan dentro de la carpeta `src/__tests__` del proyecto, manteniendo la misma estructura de carpetas que el código fuente para facilitar la navegación:

```
src/
  __tests__/
    Pages/
      home.test.jsx
      Carrito.test.jsx
      Login-Register.test.jsx
    Components/
      navegacion.test.jsx
      Productos.test.jsx
    Hooks/
      UseCart.test.jsx
    Components/
      navegacion.jsx
      Productos.jsx
    ...
  Pages/
    home.jsx
    Carrito.jsx
    ...
```

Alternativa recomendada: colocar los archivos de prueba junto a los componentes que prueban (ej: Componentes/Productos.jsx y Componentes/Productos.test.jsx). Elija la convención que prefiera y manténgala consistente.

Variables de Entorno (.env)

El proyecto utiliza las siguientes variables de entorno para las APIs. En las pruebas, estas URL se simulan con mocks:

- VITE_API_1=/MyMDentalCommerce/products/clientProducts/page
- VITE_API_2=/MyMDentalCommerce/products/getMaxProductPages
- VITE_API_3=/MyMDentalCommerce/products/filterClientProductsByPage
- VITE_API_4=/MyMDentalCommerce/products/getMaxProductPagesByDepartment
- VITE_API_5=/MyMDentalCommerce/products/getClientProductById
- VITE_API_6=/MyMDentalCommerce/Reserved/saveNewReserved
- VITE_API_7=/MyMDentalCommerce/Reserved/getReservedById
- VITE_API_8=/MyMDentalCommerce/Reserved/checkReserved
- VITE_API_9=/MyMDentalCommerce/users/findbyemail
- VITE_API_10=/MyMDentalCommerce/users/updatePerfil
- VITE_API_GetUsers=/MyMDentalCommerce/users/getUsers
- VITE_API_DEPARTMENTS=/MyMDentalCommerce/departments/getDepartments
- VITE_API_PRODUCTS=/MyMDentalCommerce/products
- VITE_API_USER_REGISTER=/MyMDentalCommerce/session/register
- VITE_API_USER_LOGIN=/MyMDentalCommerce/session/login
- VITE_API_UPDATE_USER=/MyMDentalCommerce/users/updateUser
- VITE_API_DELETE_USER=/MyMDentalCommerce/users/deleteUser
- VITE__API_GET_ALL_RESERVED=/MyMDentalCommerce/Reserved/getAllReserved

5. Conceptos Fundamentales de Jest

Jest es un framework de pruebas completo para JavaScript. A continuacion se explican los conceptos esenciales necesarios para escribir pruebas efectivas.

Bloques de prueba

`describe()`: Agrupa pruebas relacionadas en un bloque logico. Sirve para organizar el codigo de prueba y proporcionar contexto sobre lo que se esta probando.

```
describe("Componente Productos", () => {  
  // Pruebas relacionadas con Productos  
  test("deberia renderizar el titulo", () => { ... });  
  test("deberia mostrar productos", () => { ... });  
});
```

`test()` / `it()`: Define un caso de prueba individual. Ambos son alias y funcionan exactamente igual. El primer argumento es una descripcion del caso, y el segundo es la funcion que contiene las aserciones.

```
test("deberia sumar correctamente", () => {  
  expect(1 + 1).toBe(2);  
});  
  
it("deberia restar correctamente", () => {  
  expect(5 - 3).toBe(2);  
});
```

Aserciones (expect matchers)

Las aserciones verifican que el resultado de una operacion sea el esperado:

```
expect(element).toBeInTheDocument(); // Verifica que existe en el DOM  
expect(element).toHaveTextContent('Hola'); // Verifica contenido textual  
expect(element).toBeVisible(); // Verifica visibilidad  
expect(element).toHaveClass('activo'); // Verifica clase CSS  
expect(funcion).toHaveBeenCalled(); // Verifica que se llamo  
expect(valor).toEqual({ a: 1 }); // Comparacion profunda de objetos
```

Hooks de ciclo de vida

Jest proporciona funciones que se ejecutan en momentos especificos del ciclo de vida de las pruebas:

```
beforeEach(() => { /* se ejecuta antes de cada test */ });  
afterEach(() => { /* se ejecuta despues de cada test */ });  
beforeAll(() => { /* se ejecuta una vez antes de todos */ });  
afterAll(() => { /* se ejecuta una vez despues de todos */ });
```

Mocks y Simulaciones

Los mocks permiten simular funciones, modulos o APIs completas para aislar el componente bajo prueba de sus dependencias externas:

```
// Mock de una funcion  
const mockFn = jest.fn();  
mockFn();  
expect(mockFn).toHaveBeenCalled();  
  
// Mock de un modulo completo  
jest.mock("../Service/GetProductsService");
```

```
// Mock de fetch global
global.fetch = jest.fn(() =>
  Promise.resolve({ json: () => Promise.resolve([]) })
);
```


6. Definicion de Funciones y APIs de Testing

Funciones de React Testing Library

render(componente)

Renderiza un componente React en un contenedor DOM simulado. Devuelve un objeto con metodos de consulta (getByText, getByRole, getAllByText, etc.) que permiten interactuar con el componente renderizado.

screen.getByText('texto')

Busca un elemento en el DOM que contenga el texto especificado. Lanza un error si no encuentra el elemento o si encuentra mas de uno. Es la forma mas comun de localizar elementos para verificar su existencia.

screen.getByRole('button', { name: '...' })

Busca un elemento por su rol ARIA (button, heading, link, etc.) y opcionalmente por su nombre accesible. Recomendada por ser la mas cercana a como los usuarios interactuan con la aplicacion.

screen.getByTestId('id')

Busca un elemento por su atributo data-testid. Util como ultimo recurso cuando no hay forma de localizar el elemento por texto o rol.

fireEvent.click(elemento)

Dispara un evento DOM sobre un elemento. Sirve para simular clics, cambios de texto, envio de formularios, etc. Es la forma basica de simular interacciones.

userEvent.click(elemento)

Alternativa mas avanzada a fireEvent. Simula la interaccion de forma mas realista, disparando multiples eventos en el orden correcto (mouseDown, mouseUp, click, etc.). Requiere @testing-library/user-event.

waitFor(() => expect(...))

Espera de forma asincrona hasta que la asercion pasada como callback se cumpla. Util para componentes que realizan peticiones asincronas o actualizaciones de estado que toman tiempo en reflejarse en el DOM.

act(() => { ... })

Wrapper que asegura que todas las actualizaciones de estado y efectos de React se procesen antes de continuar. React Testing Library lo usa internamente, pero en casos avanzados puede necesitarse explicitamente.

Matchers Personalizados de jest-dom

La libreria `@testing-library/jest-dom` extiende `expect` con matchers especificos para el DOM, facilitando aserciones mas expresivas:

- `toBeInTheDocument()` -> Verifica que un elemento existe en el DOM
- `toBeVisible()` -> Verifica que un elemento es visible (no tiene `display:none`, `visibility:hidden`, etc.)
- `toBeDisabled()` -> Verifica que un elemento (input, button) esta deshabilitado
- `toHaveTextContent(text)` -> Verifica que un elemento contiene el texto especificado
- `toHaveClass(clase)` -> Verifica que un elemento tiene una clase CSS especifica
- `toHaveAttribute(attr, val)` -> Verifica que un elemento tiene un atributo con cierto valor
- `toHaveValue(val)` -> Verifica que un input/select tiene un valor especifico
- `toHaveLength(n)` -> Verifica que una lista de elementos tiene una longitud especifica

7. Ejemplos de Pruebas

A continuacion se presentan ejemplos progresivos, desde los mas simples hasta los mas complejos, mostrando diferentes tecnicas de prueba.

Ejemplo 1: Prueba de renderizado basico (Loader)

Verificar que un componente Loader se renderiza correctamente con su texto.

```
// src/Components/Loader.jsx
// Renderiza un spinner con el texto "Cargando..."

import { render, screen } from "@testing-library/react";
import Loader from "../Components/Loader";

describe("Componente Loader", () => {
  test("deberia mostrar el mensaje de carga", () => {
    render(<Loader />);
    expect(screen.getByText("Cargando...")).toBeInTheDocument();
  });

  test("deberia tener un elemento con rol status", () => {
    render(<Loader />);
    expect(screen.getByRole("status")).toBeInTheDocument();
  });
});
```

Ejemplo 2: Prueba de props (Productos)

Verificar que el componente Productos recibe props y las renderiza correctamente.

```
import { render, screen } from "@testing-library/react";
import Productos from "../Components/Productos";

const mockProducts = [
  { idProduct: 1, nameProduct: "Cepillo Dental", priceProduct: 25000 },
  { idProduct: 2, nameProduct: "Pasta Dental", priceProduct: 15000 },
];

describe("Componente Productos", () => {
  test("deberia renderizar la lista de productos", () => {
    render(<Productos products={mockProducts} />);
    expect(screen.getByText("Cepillo Dental")).toBeInTheDocument();
    expect(screen.getByText("Pasta Dental")).toBeInTheDocument();
  });

  test("deberia mostrar el precio formateado", () => {
    render(<Productos products={mockProducts} />);
    expect(screen.getByText(/25000/)).toBeInTheDocument();
  });
});
```

Ejemplo 3: Prueba de eventos (fireEvent en Login-Register)

Simular la interaccion del usuario en el formulario de inicio de sesion.

```
import { render, screen, fireEvent } from "@testing-library/react";
import { BrowserRouter } from "react-router-dom";
import LoginRegister from "../Pages/Login-Register";

describe("Pagina Login-Register", () => {
  test("deberia cambiar entre login y registro", () => {
    render(
      <BrowserRouter>
        <LoginRegister />
      </BrowserRouter>
    );

    // Verificar que estamos en login por defecto
    expect(screen.getByText("Iniciar Sesion")).toBeInTheDocument();

    // Hacer clic en "Registrarse"
    fireEvent.click(screen.getByText("Registrarse"));

    // Verificar que cambio al formulario de registro
    expect(screen.getByText("Crear Cuenta")).toBeInTheDocument();
  });

  test("deberia mostrar error con campos vacios", () => {
    render(
      <BrowserRouter>
        <LoginRegister />
      </BrowserRouter>
    );
    fireEvent.click(screen.getByRole("button", { name: /ingresar/i }));
    // Verificar que aparecen mensajes de error
    // (depende de la validacion implementada)
  });
});
```

Ejemplo 4: Prueba con mock de API (Home)

Simular una llamada a la API de productos para probar la pagina principal.

```
import { render, screen, waitFor } from "@testing-library/react";
import { BrowserRouter } from "react-router-dom";
import Home from "../Pages/home";

// Mock del servicio de productos
jest.mock("../Service/GetProductsService", () => ({
  getProducts: jest.fn(() =>
    Promise.resolve([
      {
        idProduct: 1,
        nameProduct: "Hilo Dental",
        priceProduct: 12000,
        urlimage: "/img/hilo.jpg",
        nameDepartment: "Higiene"
      }
    ])
  )
}));
```

```
)
});

describe("Pagina Home", () => {
  test("deberia cargar y mostrar productos", async () => {
    render(
      <BrowserRouter>
        <Home />
      </BrowserRouter>
    );

    // Esperar a que los datos se carguen
    await waitFor(() => {
      expect(screen.getByText("Hilo Dental")).toBeInTheDocument();
    });
  });
});
```

Ejemplo 5: Prueba de navegacion (Carrito con contexto)

Probar el carrito de compras que depende de un contexto global.

```
import { render, screen, fireEvent } from "@testing-library/react";
import { BrowserRouter } from "react-router-dom";
import { CartProvider } from "../Context/CartContext";
import Carrito from "../Pages/Carrito";

// Mock del contexto del carrito
const mockCart = [
  {
    idProduct: 1,
    nameProduct: "Enjuague Bucal",
    priceProduct: 18000,
    quantity: 2,
  }
];

// Mock del hook useCarrito
jest.mock("../Hooks/UseCart", () => ({
  useCart: () => ({
    cart: mockCart,
    addToCart: jest.fn(),
    removeFromCart: jest.fn(),
    totalPrice: jest.fn(() => 36000),
    makePurchase: jest.fn(),
    clearCart: jest.fn(),
  }),
}));

describe("Pagina Carrito", () => {
  test("deberia mostrar productos en el carrito", () => {
    render(
      <BrowserRouter>
        <CartProvider>
          <Carrito />
        </CartProvider>
      </BrowserRouter>
    );

    expect(screen.getByText("Enjuague Bucal")).toBeInTheDocument();
    expect(screen.getByText(/36000/)).toBeInTheDocument();
  });

  test("deberia poder eliminar un producto", () => {
    const removeMock = jest.fn();
    jest.mock("../Hooks/UseCart", () => ({
      useCart: () => ({
        cart: mockCart,
        removeFromCart: removeMock,
        // ... otros metodos
      }),
    }));

    render(
      <BrowserRouter>
```

```
    <CartProvider>
      <Carrito />
    </CartProvider>
  </BrowserRouter>
);

const deleteBtn = screen.getByRole("button", { name: /eliminar|borrar/i });
fireEvent.click(deleteBtn);
expect(removeMock).toHaveBeenCalled();
});
});
```

Ejemplo 6: Prueba completa de flujo de autenticacion

Ejemplo avanzado que simula un flujo completo de inicio de sesion con validacion de campos y redireccionamiento.

```
import { render, screen, fireEvent, waitFor } from "@testing-library/react";
import { BrowserRouter, MemoryRouter } from "react-router-dom";
import LoginRegister from "../Pages/Login-Register";
```

```
// Mock de fetch para login
global.fetch = jest.fn((url) => {
  if (url.includes("/session/login")) {
    return Promise.resolve({
      ok: true,
      json: () => Promise.resolve({
        token: "fake-token-123",
        role: "CLIENT",
        email: "test@test.com",
      }),
    });
  }
  return Promise.reject(new Error("URL no mockeada"));
});
```

```
describe("Flujo de autenticacion", () => {
  test("login exitoso redirige al home", async () => {
    render(
      <MemoryRouter initialEntries={['/acceder']}>
        <LoginRegister />
      </MemoryRouter>
    );

    fireEvent.change(screen.getByPlaceholderText(/correo/i), {
      target: { value: "test@test.com" },
    });
    fireEvent.change(screen.getByPlaceholderText(/contraseña/i), {
      target: { value: "123456" },
    });

    fireEvent.click(screen.getByRole("button", { name: /ingresar/i }));

    await waitFor(() => {
      expect(localStorage.getItem("token")).toBe("fake-token-123");
    });
  });
});
```

8. URLs de Prueba y Repositorio

Repositorio GitHub

El código fuente del proyecto se encuentra alojado en GitHub:

```
https://github.com/Duocantonio/mymdental
```

Puede clonar el repositorio con el siguiente comando:

```
git clone https://github.com/Duocantonio/mymdental.git
```

URL Base de la API

La API backend se encuentra desplegada en la siguiente URL (configurada en vite.config.js mediante proxy):

```
http://54.88.96.49:8080
```

Todas las rutas de API se resuelven a través del proxy de Vite usando las variables de entorno VITE_API_*. Por ejemplo, una petición a VITE_API_1 se traduce internamente como:

```
fetch(`/MyMDentalCommerce/products/clientProducts/page?page=${pageNum}`)  
// El proxy de Vite redirige a:  
// http://54.88.96.49:8080/MyMDentalCommerce/products/clientProducts/page?page=...
```

Endpoints Principales para Pruebas

A continuación se listan los endpoints más relevantes para simular en las pruebas:

Categoría	Endpoint
Productos	/products/clientProducts/page - Paginación de productos
Productos	/products/getClientProductById - Detalle de producto
Productos	/products/filterClientProductsByPage - Filtro por departamento
Auth	/session/login - Inicio de sesión
Auth	/session/register - Registro de usuario
Usuarios	/users/findbyemail - Buscar usuario por email
Usuarios	/users/updatePerfil - Actualizar perfil
Carrito	/Reserved/saveNewReserved - Guardar compra
Carrito	/Reserved/getAllReserved - Obtener todas las reservas
Departamentos	/departments/getDepartments - Listar departamentos

9. Criterios de Aceptación

Para considerar que la ejecución de pruebas fue exitosa, se deben cumplir los siguientes criterios de aceptación. Estos criterios garantizan que las pruebas son confiables, mantenibles y cubren los aspectos fundamentales de la aplicación.

CA-01: Cobertura mínima del 70%

Las pruebas deben cubrir al menos el 70% del código fuente, incluyendo componentes, hooks y servicios. Se puede medir con: `npm run test:coverage`

CA-02: Todas las pruebas pasan

El comando `npm test` debe ejecutar todas las pruebas sin errores ni fallos. No se aceptan pruebas con warnings de `react-dom/act()` sin resolver.

CA-03: Sin console.log en produccion

Las pruebas deben verificar que no hay console.log en el codigo de produccion (excepto en modulos de depuracion explicita).

CA-04: Componentes principales probados

Los siguientes componentes deben tener al menos una prueba cada uno:

- Loader (renderizado basico)
- Productos (renderizado con props)
- navegacion (renderizado de enlaces)
- Login-Register (cambio de formulario, eventos)
- Carrito (visualizacion y eliminacion de items)
- Home (carga de productos desde API mockeada)

CA-05: Pruebas de eventos de usuario

Al menos 3 pruebas deben utilizar fireEvent o userEvent para simular interacciones del usuario (clics, cambios en inputs, envio de formularios).

CA-06: Pruebas asincronas

Al menos 2 pruebas deben manejar operaciones asincronas usando waitFor para verificar resultados de llamadas API simuladas.

CA-07: Mocking de dependencias externas

Todas las llamadas a API deben ser mockeadas. Ninguna prueba debe hacer peticiones reales a la red. Verificar con: --verbose que no hay conexiones.

CA-08: Independencia entre pruebas

Cada prueba debe poder ejecutarse de forma independiente sin depender del estado dejado por otras pruebas. Los mocks deben resetearse entre pruebas usando beforeEach con jest.clearAllMocks().

CA-09: Pruebas de accesibilidad basica

Verificar que los elementos interactivos tienen roles ARIA apropiados (getByRole en lugar de getByTestId siempre que sea posible).

CA-10: Documentacion de pruebas complejas

Las pruebas que cubren flujos complejos (login, carrito) deben incluir comentarios explicativos de las etapas del flujo que se esta probando.

Checklist de Verificacion

Marque cada item al completarlo para garantizar que la ejecucion fue correcta:

- ☐ npm test se ejecuta sin errores
- ☐ Cobertura $\geq 70\%$ (npm run test:coverage)
- ☐ Loader se renderiza correctamente
- ☐ Productos recibe props y las muestra
- ☐ Login-Register cambia entre formularios
- ☐ Home carga productos desde API mockeada
- ☐ Carrito muestra items y permite eliminar
- ☐ fireEvent/userEvent funciona en 3+ pruebas
- ☐ waitFor se usa en pruebas asincronas
- ☐ fetch/mock no hace peticiones reales
- ☐ jest.clearAllMocks() en beforeEach
- ☐ Roles ARIA usados en lugar de testId

IMPORTANTE: Si todos los criterios de aceptacion se cumplen y el checklist esta completo, la ejecucion de pruebas se considera APROBADA.

10. Conclusion

Esta guia ha presentado un recorrido completo desde la instalacion del entorno de pruebas con Jest y React Testing Library hasta la implementacion de casos de prueba avanzados para el proyecto MyMDental.

Recordar los principios fundamentales de las pruebas en React:

- Probar comportamiento, no implementacion. Enfocarse en lo que el usuario ve y hace.
- Aislar componentes mediante mocks. No depender del backend ni de otros componentes.
- Mantener las pruebas simples y legibles. Una prueba debe contar una historia clara.
- Ejecutar pruebas frecuentemente. Integrarlas en el flujo de desarrollo (CI/CD).
- La cobertura es una guia, no un objetivo. Priorizar calidad sobre cantidad.

MyMDental - Cuidando tu sonrisa con tecnologia

<https://github.com/Duocantonio/mymdental>

Documento generado el Mayo 2026 - Version 1.0